

TestVDB: Source-Grounded Falsification of LLM-Derived Behavioral Claims for API-Conformance Testing of Vector Databases

Authors TBD

tbd@example.org

TBD (anonymity depends on venue)

TBD

Abstract

Vector Database Management Systems (VDBMSs) store the embeddings that retrieval-augmented Large Language Model (LLM) applications depend on, yet a large class of their defects lacks a practical oracle. We study *API conformance defects*: cases where a VDBMS silently accepts an input or behavior that violates its API documentation, the natural-language description of intended behavior, such as accepting out-of-range index parameters (`nprobe=0`, `ef=0`) or returning success where the documentation prescribes rejection. These defects corrupt query semantics; the large majority produce no crash, so fuzzers miss them.

Of the 111 issues TestVDB surfaced (38 maintainer-acknowledged as defects), about 85% are conformance defects that differential, metamorphic, and property-based oracles cannot reach; this is the composition of our findings, not a population estimate. The reason is structural: these accept/reject decisions cannot be mechanically checked, because the documented boundary is natural-language rather than formal. We therefore adopt an LLM as the semantic judge; prior REST-API oracle work avoids this setting by keeping deterministic, executable assertions. The judge's errors split in two. Family-specific self-preference is mitigated by cross-model validation. *Task-intrinsic* errors, where ambiguous documentation makes different LLM families infer the same wrong claim, are not: in a twelve-clause pilot on Milvus and Qdrant, a different family reproduced GLM's over-strict claim in 5, and the over-strict phenomenon concentrates in optional-default APIs (none on Weaviate, whose docs state explicit bounds). The implementation, as the most accessible source of actual behavior, is what resolves them.

We instantiate this as TestVDB, which falsifies LLM-derived behavioral claims against source. TestVDB surfaced 111 candidate issues across five VDBMSs; maintainers acknowledged 38 as defects, and a controlled retrospective on Milvus and Qdrant shows the source anchor suppresses 81% of false positives (up from 31%) while retaining 96.7% of true positives.

CCS Concepts

• **Software and its engineering** → **Software testing and debugging**.

Keywords

Vector database, API conformance testing, test oracle, LLM-derived oracle, LLM-as-judge, source-grounded falsification

1 Introduction

Vector Database Management Systems (VDBMSs) store the embeddings that retrieval-augmented LLM applications depend on. Their defects are costly: an empirical study [?] finds that more than half of VDBMS bugs manifest as functional failures, and the VDBMS testing roadmap [?] attributes about 43% of VDBMS bugs to incorrect behavior and identifies oracle definition as a key challenge. Existing VDBMS fuzzers [?] detect only crashes. We target a prevalent subset: *API conformance defects*, where a VDBMS silently accepts an input or produces a behavior that violates its API documentation, such as accepting `nprobe=0` or out-of-range index parameters such as `ef=0`, or returning success where the documentation prescribes rejection. These defects corrupt query semantics and can return data the documentation intended to exclude (for example, a negative score threshold that disables a filter and returns all matches). Most produce no crash (37 of 38 acknowledged in our data).

The oracle problem [?] is acute here. Across 111 submitted VDBMS issues (38 maintainer-acknowledged as defects), about 85% are, by our classification, conformance defects that the standard oracle families cannot reach (Table 1). Differential testing catches mathematical-invariant violations across vendors but cannot adjudicate accept/reject, which diverges across VDBMSs by design and leaves no cross-vendor reference; the roadmap flags differential testing as challenging in this setting [?]. Metamorphic relations [?] address result correctness but not input-acceptance decisions, because a metamorphic relation is an output relation, not an input-acceptance transform. Property-based testing needs a machine-checkable property and an OpenAPI schema; VDBMS endpoints serve no schema that encodes these conformance constraints, and several serve no OpenAPI at all. The root cause is that many of these accept/reject decisions cannot be mechanically checked, because the documented boundary is natural-language and ambiguous rather than formal.

The absence of a mechanical oracle for these cases places VDBMS conformance in a setting where the LLM serves as the semantic oracle (Section 3): an LLM must both extract behavioral claims from the documentation and, where extraction is impossible, judge

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnnn.nnnnnnn>

Table 1: Oracle candidates by the defect class they reach, and why the API-conformance residual leaves an LLM as the practical oracle. Rows 1 to 4 use a deterministic oracle; row 5 (REST doc/spec-derived assertion oracles) also keeps a deterministic oracle and so never enters this regime; only row 6 does.

Candidate oracle	Reaches (defect class)	Why it misses the conformance residual
Crash (VDBFuzz [?])	crash/hang	most conformance defects do not crash (37/38 acknowledged)
Differential testing	math invariants across vendors (our model-free subclass, §6)	cross-vendor accept/reject diverges by design; no conformance reference [?]
Metamorphic relations [?]	result correctness (top- k monotonicity, recall vs. ef)	an MR is an output relation; conformance is an input accept/reject decision, and no transform preserves it
Property-based testing [? ? ?]	math properties + schema conformance	needs a machine-checkable property and an OpenAPI schema; VDBMS endpoints serve no schema that encodes these constraints, and several serve no OpenAPI
REST doc/spec-derived oracles (AGORA+ [?], SATORI [?], MASTOR [?])	status/field/invariant assertions from OpenAPI, traces, or source	reliable extraction from low-ambiguity structured sources; no falsification needed
LLM-derived oracle (TestVDB)	accept/reject vs. API documentation	many conformance semantics are not mechanically checkable, leaving an LLM as the practical oracle for the residual; the reliability problem this introduces is the subject of §3

conformance directly. The gap is in source ambiguity: prior REST-API oracle work [? ? ?] extracts from structured sources where constraints are explicit, while VDBMS documentation is ambiguous and LLM interpretation may produce wrong claims. The errors this introduces split into a family-specific layer that cross-model validation mitigates and a *task-intrinsic* layer it cannot, because the ambiguity is in the documentation. *Source-grounded falsification* (Section 4) resolves the task-intrinsic layer by treating these claims as refutable hypotheses and checking them against the implementation’s actual behavior. We instantiate it in TestVDB, a detector that extracts behavioral claims from documentation, probes VDBMS endpoints, and falsifies LLM verdicts against source.

Contributions.

- **The role of the LLM (§3).** We show that VDBMS documentation, unlike structured specifications, forces an LLM into both extraction and judgment roles, that the resulting behavioral claims are unreliable in two distinct ways (family-specific and task-intrinsic), and that this source-ambiguity gap (low-ambiguity sources yield reliable assertions, whereas ambiguous natural-language documentation yields claims that may be wrong) is what separates VDBMS conformance testing from prior REST-API oracle work.
- **Quantification of the conformance residual.** About 85% of the conformance defects we found are unreachable by differential, metamorphic, or property-based oracles, mapping where each classical class fails.
- **Task-intrinsic documentation-interpretation errors and the source-grounded counter.** We separate family-specific from task-intrinsic documentation-interpretation errors and show, on a twelve-clause probe on Milvus and Qdrant, that cross-model judging misses 2 of the 5 task-intrinsic clauses while source-grounded falsification catches all 12, and that over-strict concentrates in optional-default APIs (Milvus and Qdrant’s search parameters) and is absent

where documentation states explicit bounds (Weaviate, whose doc-code gaps are conformance bugs instead).

- **TestVDB and scale.** TestVDB surfaced 111 candidate issues across five VDBMSs; maintainers acknowledged 38 as defects. A controlled retrospective on Milvus and Qdrant shows the source anchor suppresses 81% of false positives (up from 31%) at 96.7% true-positive retention.
- **A reusable model-free invariant oracle subclass** (COSINE bounds, index completeness) that is classical-addressable, cross-vendor, and independent of TestVDB’s LLM pipeline.

2 Background and Problem Setup

Barr et al. [?] classify test oracles by the artifact that supplies the pass/fail verdict: *specified* (from a specification or contract), *derivable* (from execution, as in differential, metamorphic, or invariant testing), *implicit* (a crash), or *none* (a human). TestVDB uses two: a documentation-derived oracle for conformance defects, in the specified class¹, and a model-free invariant oracle for mathematical-bound violations, in the derivable class. The subset we care about is conformance, and the central observation is that on this subset the first three classes fail, leaving only the human oracle, or its scalable proxy, an LLM.

Conformance testing originated in protocol standards [?]; in the API setting it checks whether an implementation matches its documented contract, and REST API conformance defects are a recognized fault class [?]. We separate conformance from correctness. *Conformance* asks whether the API’s accept/reject behavior matches its documentation. *Correctness* asks whether a returned result is mathematically right, such as approximate nearest neighbor (ANN) recall or ranking. TestVDB targets conformance; result correctness of vector search remains open and is not our claim [?].

For a substantial portion of conformance cases, the documented boundary is natural-language prose rather than a formal grammar,

¹VDBMS API documentation, unlike the formal assertions of Design by Contract [?], is natural-language prose; this informality is the source of both the need for an LLM and the reliability problem we address.

so the verdict cannot be mechanically checked without semantic interpretation. We formalize the consequence in Section 3.

3 The Role of the LLM and Its Reliability Problem

VDBMS API documentation, unlike the structured sources that REST-API oracle tools rely on (OpenAPI specifications, execution traces, or source code), is natural-language prose. This forces an LLM into the testing pipeline in two ways. First, the LLM must extract behavioral claims from the documentation, translating vague descriptions such as “optional, default 1” into checkable clauses such as “parameter ≥ 1 .” Second, for inputs whose treatment the documentation leaves entirely unstated, the LLM must judge conformance directly, because no extractable clause exists. In both roles, the LLM’s output is a behavioral claim that may be wrong: the extraction may over-formalize the documented intent, and the direct judgment may hallucinate. A human can perform these roles but does not scale, so in practice both extraction and judgment fall to a language model. This setting combines extraction from ambiguous documentation with, where extraction fails, direct semantic judgment; in both roles, the oracle is LLM-derived and therefore unreliable.

The contrast with recent REST-API oracle work is about source ambiguity, not extraction mechanism. AGORA+ [?], SATORI [?], and MASTOR [?] extract oracles from structured sources (OpenAPI specifications, execution traces, or source code) where the constraints are explicit and the LLM’s task is to transcribe them into assertions. Even though SATORI and MASTOR also use LLMs, the low ambiguity of their sources means the extracted assertions are reliable and false-positive rates are low. TestVDB extracts from natural-language documentation, where the constraints are implicit and ambiguous and the LLM’s task is to interpret intent. Interpretation from ambiguous text is unreliable: the LLM may stably produce a wrong claim: the same wrong claim across runs and across model families, because the ambiguity is in the shared input, not the model. This stable misinterpretation is the gap that source-grounded falsification addresses. MASTOR is the closest in using source, but it tests what the implementation does, with source as the reference, and so cannot detect a gap between the documentation and the code; TestVDB tests what the documentation prescribes, with source as the reference for actual behavior, and that gap is exactly what it targets.

Entering this setting exposes a reliability problem with two layers. The first is family-specific: when one LLM family both extracts behavioral claims from the documentation and judges conformance, the two roles share biases, and the judge tends to confirm the extractor’s errors. This is the LLM-as-judge self-preference phenomenon (Panickssery et al. [?]), and cross-model validation, in which a second family judges, mitigates it. The second layer is deeper. When the documentation itself is ambiguous, different LLM families extract the same wrong claims. For example, a parameter documented as “optional, default 1” is over-formalized by both families as “must be at least 1,” when the real semantics is that the value 0 selects the default. The error originates in the shared input rather than in either model, so cross-model validation cannot remove it.

We scope the stability claim precisely. Task-intrinsic stability is an extraction-level property defined across model families: a clause is task-intrinsic when a second family’s independent formalization of the same documentation is also over-strict on the same parameter. This is distinct from the intra-judge self-inconsistency that Haldar et al. [?] report for LLM-as-a-judge ratings, where a single judge’s verdict on the same input varies across runs—a sampling-noise phenomenon in the judgment step. Our claim concerns extraction: when the documentation is ambiguous, different families converge on the same wrong clause, and that convergence is what cross-model validation cannot break.

We confirmed the split with two probes on twelve over-strict clauses (nine from Milvus, three from Qdrant v1.18.2, live-probed to confirm over-strict). We first fed each raw documentation passage to two LLM families, GLM and DeepSeek, and asked each to formalize the documented behavior independently; DeepSeek reproduced an over-strict clause on the same parameter in 5 of the 12 (2 of 9 Milvus, 3 of 3 Qdrant), the task-intrinsic subset; a clause counts as task-intrinsic when the second family’s independent formalization is also over-strict on the same parameter, a comparison at the parameter level rather than verbatim. We then tested the operational alternative directly: we asked DeepSeek to judge each of GLM’s over-strict clauses against the documentation. Cross-model judging caught 7 of the 12 but missed 2 of the 5 task-intrinsic ones, where DeepSeek read the ambiguous documentation as GLM had. Source-grounded falsification, by contrast, contradicts every over-strict clause, because the implementation accepts the value the clause rejects. The probe is a twelve-clause pilot, and we treat it as pending a larger study (Section 6).

The implication is the basis for our method. Cross-model validation covers the family-specific subset but not the task-intrinsic one. The task-intrinsic subset requires a source of actual behavior that does not depend on the documentation; the most accessible one is the implementation. This motivates source-grounded falsification, developed in Section 4: treat the LLM-derived behavioral claims as refutable hypotheses and falsify them against source, where the documentation expresses the intended behavior and the source expresses the actual behavior.

4 TestVDB Design

TestVDB instantiates source-grounded falsification. It treats LLM-derived behavioral claims as a set of clauses, each a hypothesis that the API enforces some documented constraint, and uses the implementation’s actual behavior to falsify them.

The falsification rule is concrete. For a clause that asserts the API rejects inputs where a parameter violates a constraint (for example, shardsNum ≥ 1), the dev-reviewer (the pipeline’s source-reading agent) examines the implementation. If the source shows that shardsNum = 0 selects the default shard number, the clause is over-strict and the verdict that relied on it is falsified, a false positive suppressed. If the source shows no such intended semantics yet the implementation still accepts a value the documentation prescribes rejecting, the violation stands as a real defect.

This use of source is the opposite of MASTOR’s [?]: MASTOR reads source to generate oracles that encode implemented behavior and so cannot detect a gap between the documentation and the code,

whereas TestVDB reads source to falsify documentation-derived clauses and targets exactly that gap (Section 7).

The pipeline has five stages. An LLM first extracts behavioral claims from the documentation as clauses. Attack agents then generate boundary inputs targeting each clause. An LLM-derived oracle judges whether each observed response conforms to the documentation; this is the step whose two-layer unreliability we described in Section 3. The dev-reviewer then falsifies the clauses and the LLM’s verdicts against source. A novelty gate finally removes duplicates and known issues.

The dev-reviewer applies three anchors: a clean reproduction that re-probes the live API, a source-grounded anchor that is the primary falsifier, and an optional threat-model cross-check that compares each candidate against maintainer-acknowledged by-design and wont-fix patterns to suppress known non-defects. This separation matters because the same LLM family produces both the behavioral claims and the verdicts; the implementation, an independent information source, is what falsifies them.

5 Implementation

TestVDB runs as a multi-agent pipeline on the Claude Code runtime, with each of the 20 agents defined by a task-structured role prompt and dispatched to a GLM-5.2 backbone (via the bigmodel Anthropic-compatible API) under the Claude Code runtime’s default sampling, with no fixed random seed; we have not measured run-to-run variance and flag this as a limitation. Target VDBMSs run in pinned Docker versions (Milvus 2.6.19 for the single-LLM and single-layer ablations; the full version matrix is in the artifact). A complete run per target exercises the 20 agents over tens of generated candidates; wall-clock is dominated by the dev-reviewer’s source-grounding step, repository clone and source retrieval, and live Docker re-probes, rather than by raw LLM latency. The 111-submission study plus the evaluation batches amount to on the order of 10^4 LLM calls and roughly \$10 per target at current pricing, comparable to a few hours of manual boundary testing. The full prompts, target versions, and per-token accounting are in the artifact.

6 Evaluation

We evaluate TestVDB on five VDBMSs (Milvus, Qdrant, Weaviate, MeiliSearch, Chroma) and ask four questions. **RQ1**: how many conformance defects does TestVDB surface, and how large is the residual beyond classical oracles? **RQ2**: does source-grounded falsification suppress false positives? **RQ3**: can cross-model validation resolve task-intrinsic documentation-interpretation errors, or is source needed? **RQ4**: does the model-free invariant subclass find real bugs on its own?

RQ1: yield and the conformance residual. TestVDB surfaced 111 candidate issues across the five systems; maintainers acknowledged 38 as defects (31 fixed, 7 accepted-open). The remaining 73 comprise 12 by-design, 4 rejected, 24 duplicates, and 33 pending or closed. Submissions span Milvus (51), Weaviate (30), and Qdrant (26), but maintainer acknowledgment concentrates on Milvus (22) and Qdrant (13); Weaviate, MeiliSearch, and Chroma contribute breadth rather than statistical weight (Table 2). To quantify the

Table 2: Issues submitted and maintainer-acknowledged per VDBMS.

VDBMS	Submitted	Acknowledged
Milvus	51	22
Weaviate	30	3
Qdrant	26	13
MeiliSearch	3	0
Chroma	1	0
Total	111	38

residual, we classified each submitted issue by fault model: classical-addressable (a mathematical-invariant or crash that a differential, metamorphic, or crash oracle could in principle find), conformance (an accept/reject violation only a semantic judge reaches), or concurrency (a race or atomicity violation). About 85% of the issues we submitted are, by this classification, conformance defects that classical oracles cannot reach (89% on the 38 maintainer-acknowledged subset) — we emphasize this is the composition of TestVDB’s findings, not a prevalence estimate of the true defect distribution. The remainder split between classical-addressable cases (about 10%) and concurrency defects (about 5%). A full-space estimation with capture-recapture or an unbiased defect sample is future work. As a structural check, we ran a classical-oracle suite on Qdrant v1.18.2: a metamorphic relation set (distance symmetry, top- k monotonicity, the COSINE bound, self-similarity) found no violations on this version and, by construction, no conformance defects, since metamorphic relations relate outputs rather than input-acceptance decisions. The mathematical-invariant violations such checks surface are the domain of our model-free subclass (RQ4); the conformance residual lies outside classical reach. A direct head-to-head with VDBFuzz [?] confirms the complementarity empirically: on Qdrant v1.18.2, VDBFuzz executed over 26,000 mutated requests across five test templates and found 0 crashes and 0 non-200 responses, while TestVDB surfaced conformance defects on the same version. The two tools’ oracles operate on disjoint defect classes.

RQ2: effect of source-grounded falsification. On a controlled retrospective over 54 maintainer-adjudicated candidates (38 acknowledged, 12 by-design, 4 rejected), the dev-reviewer’s source-grounded anchor suppresses 81% of false positives, up from 31% with only the other two anchors (clean reproduction and threat-model cross-check), while retaining 96.7% of true positives ($n=30$). Aggregated end-to-end precision on the adjudicated pool is 69.2% (Wilson 95% CI [55.7%, 80.1%]). Of the remaining submissions, about 30 are still pending maintainer triage; the pending-resolution worst-case bound, which treats all pending as false positives, widens the interval to [43.9%, 80.5%]. Ablating the configuration shows where the precision comes from: single-LLM self-judgment (no source, no multi-agent debate) achieves 25.5%; adding a single source-grounded cycle lifts precision to 45.6%; the full multi-agent debate with the source anchor reaches 69.2%. The anchors contribute at different points. On a Milvus re-probe of 27 candidates, the clean-reproduction anchor alone kills tooling-artifact false positives at 12/14 but by-design ones at only 2/11. A three-condition ablation on a 12-FP/4-TP control (Milvus v2.6.19,

dev-reviewer-v2, blind) then isolates the dev-reviewer's two anchors: source alone suppresses 9/12 FPs (75%), the threat-model anchor alone 6/12 (50%, unstable on state/concurrency FPs), and their union 11/12 (91%), each retaining all 4 TPs, confirming the source-grounded anchor as the dominant contributor. The full per-anchor breakdown is in the artifact. As a cross-model check, DeepSeek re-running the dev-reviewer on twenty candidates across three studies—six from the retrospective plus fourteen newly reviewed blind to GLM-5.2's rationale, spanning input-validation, upsert-semantics, idempotent-drop, correct-reject, and dynamic-field subtypes—agrees with GLM-5.2 on all twenty (Cohen's $\kappa = 1.0$), so the source-grounded verdict does not appear family-specific when source evidence is explicit. RQ3 next isolates the subset of these errors that cross-model validation cannot reach, where source is the only resolver.

RQ3: cross-model validation vs source on task-intrinsic errors. The central claim is that source-grounded falsification resolves task-intrinsic documentation-interpretation errors that cross-model validation cannot. We tested this on twelve over-strict clauses: nine from GLM's Milvus formalization plus three from Qdrant v1.18.2, confirmed over-strict by live probe (timeout=0, group_size=0, and an out-of-range score_threshold are all accepted; limit=0, shard_number=0, and replication_factor=0 are rejected, so Qdrant enforces most of its documented minimums and only these three qualify). We first asked DeepSeek to formalize each passage independently; it reproduced an over-strict clause on the same parameter in 5 of the 12 (2 of 9 Milvus, 3 of 3 Qdrant), the task-intrinsic subset. We then asked DeepSeek to judge each of GLM's clauses against the documentation; cross-model judging caught 7 of the 12 but missed 2 of the 5 task-intrinsic ones, where DeepSeek read the ambiguous documentation as GLM had. Source-grounded falsification contradicts all twelve, because the implementation accepts the value each over-strict clause rejects. The task-intrinsic phenomenon is not confined to Milvus: all three Qdrant over-strict clauses are task-intrinsic. The TI rate is 5/12 (Wilson 95% CI [19%, 68%]); we treat the twelve-clause probe as a pilot pending a larger head-to-head study. This vendor-wise distribution is itself a finding: over-strict concentrates in APIs with many optional-default parameters (Milvus, Qdrant's search parameters) and is absent where documentation states explicit bounds (a parallel e2 expansion on Weaviate v1.38.2 surfaces no over-strict clauses, because Weaviate documents "Must be >= 1" for ef, dynamicEfMin, etc., so its doc-code gaps are conformance bugs rather than over-formalized clauses), which is why the twelve-clause set is Milvus-heavy and why scaling to $n=30$ is bounded by the phenomenon rather than by sampling effort. A within-vendor contrast isolates documentation style as the driver rather than the vendor: on Qdrant, the three search parameters that take optional defaults (timeout, group_size, score_threshold) are over-strict, while its collection parameters that state explicit minimums (shard_number, replication_factor, write_consistency_factor, all documented "Minimum 1") reject 0 and are not. This yields a falsifiable prediction: a VDBMS parameter documented with an optional default and no explicit bound is a candidate for over-formalization, whereas one documented with an explicit minimum is not.

Table 3: Cross-model judging vs source-grounded falsification on twelve GLM over-strict clauses (nine Milvus + three Qdrant v1.18.2, live-probe-confirmed). TI marks the five task-intrinsic clauses. Cross-model judging misses 2 of 5 TI clauses; source-grounded falsification contradicts all twelve. The behavior over-strict and explicit-bound subtypes extend this probe to $n=29$ (see text, RQ3).

Over-strict clause	TI	Cross-model judging	Source-grounded
shardsNum ≥ 1	yes	missed	caught
metricType strict enum	no	missed	caught
consistencyLevel strict enum	no	caught	caught
data non-empty	yes	missed	caught
outputFields required	no	caught	caught
limit ≥ 1	no	caught	caught
roundDecimal ≥ 0	no	caught	caught
offset ≥ 1	no	caught	caught
dbName required	no	caught	caught
timeout ≥ 1	yes	caught	caught
group_size ≥ 1	yes	missed	caught
score_thr. $\in [0, 1]$	yes	missed	caught
Total	5	7/12	12/12

A behavior-level probe extends the phenomenon beyond parameters. On four Milvus by-design behavior issues—search on an unloaded collection, duplicate collection creation, drop on a non-existent collection, and a leading-underscore collection name (all maintainer-marked resolution/by-design)—DeepSeek independently over-formalized the documented constraint on 4 of 4; cross-model judging caught none (0/4); source-grounded falsification caught all 4. The task-intrinsic phenomenon is thus not confined to optional-default parameters, though we report this behavior subtype separately to preserve the within-vendor parameter contrast above. Pooled across parameter and behavior subtypes ($n=16$), the task-intrinsic rate is 9/16 (Wilson 95% CI [33%, 77%]), and cross-model judging catches 3 of the 9.

A specificity check confirms the phenomenon does not generalize to explicit bounds. On thirteen parameters whose documentation states explicit bounds—Qdrant's shard_number, replication_factor, write_consistency_factor, full_scan_threshold; Weaviate's ef, dynamicEfMin, dynamicEfMax, efConstruction, maxConnections, vectorIndexType, replicationConfig.factor; Milvus's dimension, num_partitions—DeepSeek independently over-formalized zero (0/13, Wilson 95% CI [0%, 23%]). The task-intrinsic phenomenon concentrates in ambiguous optional-default APIs and is absent where documentation states explicit bounds, reinforcing the within-vendor contrast.

RQ4: model-free invariant oracle subclass. Separately from the LLM pipeline, a model-free invariant subclass detects violations of hard mathematical bounds: COSINE distance above 1 for identical vectors, an index returning 2 of 25 matching points, and a payload filter returning points that miss a required field. The subclass probes the API directly with boundary inputs (identical vectors, a partially indexed collection, a filter on an absent field) and checks

each response against the bound, with no documentation-based extraction or LLM judgment. These reproduce across Milvus and Qdrant and are the least design-contingent part of the evaluation. The 111 submissions include 9 mathematical-invariant issues of this class (the three above plus further bound violations), which the model-free subclass targets; they are classical-addressable and counted within the roughly 10% non-conformance portion of RQ1, not separately from the 111.

Threats to validity. *Internal validity.* The RQ3 probe covers twenty-nine clauses across three subtypes (twelve over-strict parameters, four by-design behaviors, thirteen explicit-bound negatives); DeepSeek over-formalizes at 9/16 on ambiguous optional-default APIs and 0/13 on explicit bounds. The over-strict subset remains the most contingent finding; the retrospective and the yield are the broader evidence base, and a larger head-to-head study is ongoing. *External validity.* Generalization to Weaviate, MeiliSearch, and Chroma is breadth-only; statistical claims rest on Milvus and Qdrant, and soft result correctness of vector search (ANN recall, ranking) is out of scope. *Construct validity.* All source-anchor results use a single model family (GLM-5.2); a cross-model check re-running the dev-reviewer with DeepSeek on twenty candidates (Cohen's $\kappa = 1.0$, blind to GLM-5.2's rationale, spanning input-validation, upsert-semantics, idempotent-drop, correct-reject, and dynamic-field subtypes) suggests the verdict is not strongly family-specific when source evidence is explicit; we still do not estimate recall because there is no public ground-truth defect catalog for VDBMSs.

7 Related Work

VDBMS testing. VDBFuzz [?] is the first dedicated VDBMS fuzzer and uses crash as its oracle, which makes it complementary to this work. The roadmap [?] sets the agenda we build on, and the empirical bug study [?] the taxonomy; both point to the conformance subset we target.

REST-API oracle generation. A recent line derives test oracles for REST APIs from specifications or source code. AGORA+ [?] learns dynamic invariants from request and response traces, SATORI [?] uses an LLM to read the OpenAPI specification, and MASTOR [?] uses a multi-agent system over source code. These are not in our setting: SATORI requires an OpenAPI specification and AGORA+ an executable trace against a schema, neither of which VDBMS endpoints serve, and each produces an oracle that remains an executable assertion, checked deterministically, so the LLM never issues a test verdict (Section 3). MASTOR is the closest, since it also uses source, but it tests what the implementation does, with source as the reference, and so cannot detect a gap between the documentation and the code; TestVDB tests what the documentation prescribes, with source as the reference for actual behavior, and that gap is exactly what it targets.

LLM-as-judge reliability. Self-preference bias, in which an LLM evaluator favors its own family's outputs, is established for general text evaluation (Panickssery et al. [?]; Wataoka and Takahashi [?]); we observe the analogous bias in the test-oracle pipeline. Our family-specific layer is an instance of it, and cross-model validation mitigates it as expected. A separate reliability axis is intra-judge

self-inconsistency: Halder et al. [?] show that a single LLM judge's ratings vary substantially across identical runs. Our task-intrinsic layer is orthogonal to both: when the documentation is ambiguous, different model families converge on the same wrong clause at the extraction step, so the error survives cross-model validation. Section 3 scopes this extraction-level stability claim precisely.

Database and property-based oracles. NoREC [?], TLP [?], DQE [?], and DDLCheck [?] test SQL correctness against reference semantics that VDBMS conformance lacks. Property-based testing [?] and its REST instances, Schemathesis [?] and QuickREST [?], together with metamorphic testing (MeTMAP [?]), cover the mathematical-invariant subset; the conformance residual is where they, by construction, stop.

Documentation-derived oracles. Toradocu [?] pioneered oracle extraction from natural-language documentation, using NLP and pattern matching to translate Javadoc @throws comments into executable assertions. Its extraction is deterministic but limited to simple syntactic patterns, and it acknowledges false positives from extraction failures without correcting them. Doc2OracLL [?] extended this line to LLMs, showing that documentation quality directly impacts LLM-generated oracle correctness, and AugmentTest [?] similarly infers oracles from the available documentation, but both still trust the generated oracle without verification. Konstantinou et al. [?] provide direct evidence for the risk this creates: LLM-generated oracles tend to capture the actual program behavior rather than the expected, correct behavior, which is precisely the failure mode our source-grounded falsification targets. ChatAssert [?] addresses false positives through iterative prompt repair guided by compilation and execution feedback, and Testora [?] uses natural-language PR descriptions as a regression oracle, achieving 55% precision even with a multi-question classifier, further evidence that LLM judgment alone remains unreliable. All of these treat the LLM as the final arbiter: ChatAssert and Testora verify through runtime behavior (compilation feedback, differential execution), but the LLM still makes the final call. TestVDB differs by using the implementation's source code, not runtime behavior, as an independent verification source, falsifying LLM-derived behavioral claims and targeting task-intrinsic documentation-interpretation errors that prompt refinement and runtime repair cannot reach.

8 Discussion and Limitations

This setting is not specific to VDBMSs. Any system whose documentation is natural-language prose rather than a structured specification enters it: REST APIs without OpenAPI coverage, configuration validation, and policy-as-code checks where the documentation is the only oracle. In each, a single LLM family that both extracts the specification and checks conformance is susceptible to the two-layer unreliability we describe, and source-grounded falsification is the natural mitigation. We have not tested these transfers and leave them to future work.

Beyond the threats to validity in Section 6, two limitations bound the approach itself. Source-grounded falsification requires source, so it does not transfer to closed-source VDBMSs, and it treats the implementation as correct, so an implementation bug can wrongly

falsify a clause whose documentation is right. The association between documentation style (optional-default versus explicit-bound) and over-formalization risk is a correlative observation, validated by within-vendor contrast and a falsifiable prediction (RQ3) but not a causal claim; ruling out alternative explanations such as team structure or implementation complexity is future work. The 85% residual is the composition of TestVDB's findings, biased toward conformance by design, not an estimate of the true defect distribution.

9 Conclusion

VDBMS API conformance is a defect class where a substantial portion of accept/reject decisions resist deterministic checking, which leads us to adopt an LLM for extraction and judgment. The LLM's documentation-interpretation errors split into a family-specific

layer that cross-model validation mitigates and a task-intrinsic layer that it cannot, because the ambiguity lives in the documentation. Source-grounded falsification resolves the task-intrinsic layer by treating the LLM-derived behavioral claims as hypotheses and the implementation's actual behavior as the reference. TestVDB demonstrates the approach at scale: 111 candidate issues across five VDBMSs, 38 maintainer-acknowledged, with the source anchor suppressing 81% of false positives on Milvus and Qdrant. The boundary we draw, between problems where extraction from structured sources suffices and those where natural-language documentation forces LLM interpretation, is where we see the testing of LLM-dependent systems heading.

References